Scalable Flash Sale System

Project Report

Horizontal Scaling · Distributed Consistency · Resilience Patterns · Atomic Operations

---

Dawei Feng · Rajkesh Prakash Shetty

CS6650 — Building Scalable Distributed Systems

January 2026 – April 2026

1. Introduction

This report presents four experiments evaluating the scalability, correctness, resilience, and atomicity of a flash sale system built in Go on AWS ECS Fargate with an ALB and ElastiCache Redis. Flash sales require a system to guarantee exactly 100 items sold under thousands of concurrent users — no more, no less.

2. System Architecture

A stateless Go/Gin service exposes POST /purchase, GET /health, and POST /reset. Two stock backends are supported: In-Memory (atomic CAS per instance — fast but isolated) and Redis (atomic DECR or Lua script — shared and correct). Resilience is controlled via RESILIENCE_MODE env var: none, failfast (500ms timeout), bulkhead (100 concurrent cap), or all three with a Circuit Breaker. All infrastructure is Terraform-managed: VPC, ALB, ECS Fargate, ElastiCache Redis, ECR, and CloudWatch.

3. Methodology

Experiments 1 & 2: 1,000 concurrent users, 200/s spawn rate, 60s, 100 stock. Experiments 3 & 4: 200 users primary (matching 100 stock for realistic flash sale scenario) and 5,000 users supplementary, 60s. Metrics: throughput (req/s), response time (P50/P95/P99), and correctness (exact items sold verified via /health). Stock was reset between runs via POST /reset.

4. Experiment 1: Horizontal Scaling

4.1 Objective & Setup

Does adding ECS Fargate instances behind an ALB improve throughput and latency under burst traffic while maintaining correctness? Three configurations tested (1, 2, 4 instances) with Redis backend, 1,000 concurrent users, 60 seconds.

4.2 Results

Configuration	Throughput (RPS)	Median (ms)	P95 (ms)	P99 (ms)	Items Sold	Correctness
1 Instance	1,515	500	590	870	100	PASS
2 Instances	1,509	520	590	1,000	100	PASS
4 Instances	1,533	490	580	780	100	PASS

Figure 1: Experiment 1 — Throughput, Latency, and Correctness by Instance Count

4.3 Analysis

Throughput was flat across all three configurations (~1,509–1,533 RPS) because the Locust client machine hit 90%+ CPU utilization, capping load at ~1,500 RPS regardless of server capacity. This is a client-side bottleneck — distributed Locust across multiple machines would be needed to observe true server-side scaling. Correctness held at exactly 100 items sold in every configuration. Redis DECR is the single source of truth and its atomicity holds regardless of how many instances call it simultaneously.

Key takeaway: Horizontal scaling maintains correctness. The throughput ceiling is a load-generator limitation, not a server limitation. Redis atomic DECR guarantees no overselling at any scale.

5. Experiment 2: In-Memory vs Redis

5.1 Objective & Setup

Does an in-memory counter break correctness under horizontal scaling? 4 ECS instances, 1,000 users, 60s. In-memory: per-instance atomic CAS counter. Redis: shared ElastiCache with atomic DECR.

5.2 Results

Metric	In-Memory (4 instances)	Redis (4 instances)
Throughput (RPS)	1,524	1,444
Median Latency (ms)	490	510
P95 Latency (ms)	590	640
P99 Latency (ms)	620	890
Items Sold	400 (4x oversell)	100
Correctness	FAIL	PASS

Figure 2: Experiment 2 — In-Memory vs Redis Correctness and Latency

5.3 Analysis

The in-memory backend demonstrates the fundamental correctness problem in distributed systems: each instance operates on an isolated copy of state. With 4 instances, the ALB distributes requests roughly evenly, so each instance independently sells up to 100 items — resulting in 400 total sales, a 4x oversell. The Redis backend pays a latency cost for every purchase (a network round-trip to ElastiCache, ~20ms overhead on median), but gains global correctness. The atomic DECR command ensures the total number of successful decrements never exceeds the initial stock regardless of concurrency.

Key takeaway: In a distributed flash sale system, shared state (Redis) is not optional — it is a correctness requirement. The ~5% throughput reduction and ~40% P99 increase are negligible compared to the catastrophic business impact of overselling.

6. Experiment 3: Resilience Patterns Under Load

6.1 Objective & Setup

Do resilience patterns (Fail Fast, Bulkhead, Circuit Breaker) improve throughput and tail latency under load? Four configurations tested: none, failfast (500ms timeout), bulkhead (100 concurrent cap), and all combined (with Circuit Breaker: threshold=10, cooldown=5s). Primary: 200 users vs 100 stock, 2 ECS instances, Redis. Supplementary: 5,000 users to confirm trends at extreme scale.

6.2 Results — Primary (200 users, 100 stock)

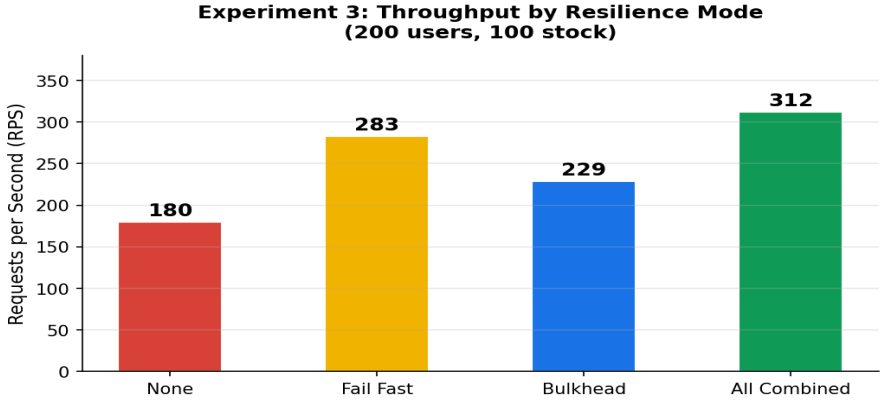

Figure 3: Throughput by Resilience Mode (200 users)

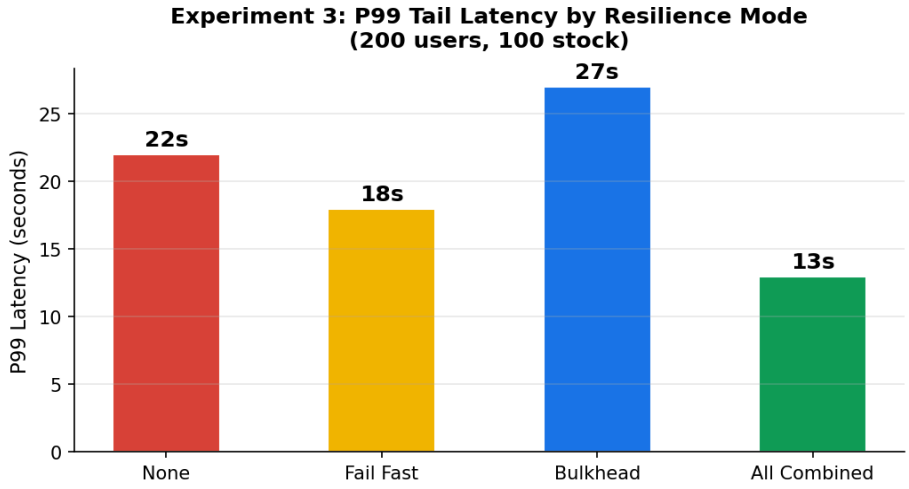

Figure 4: P99 Tail Latency by Resilience Mode (200 users)

Mode	RPS	Avg (ms)	P95 (ms)	P99 (ms)	Max (ms)
None	180	437	30	22,000	51,129
Fail Fast	283	354	31	18,000	50,118
Bulkhead	229	474	26	27,000	48,140
All	312	293	29	13,000	45,112

Figure 5: Experiment 3 Full Results (200 users)

6.3 Supplementary (5,000 users)

At extreme scale: All Patterns → 638 req/s vs 576 req/s (no protection), P99 12,000ms vs 15,000ms. Confirms the patterns remain beneficial under 50x the available stock.

6.4 Analysis

All Combined produced the best results: +73% throughput, -33% avg latency, -41% P99 vs no protection. The three patterns are complementary — Fail Fast sheds slow requests, Bulkhead prevents resource exhaustion, Circuit Breaker stops cascading failures. Together they keep the system in controlled degradation rather than uncontrolled collapse.

Key takeaway: Controlled failure is better than uncontrolled degradation. Combining all three patterns delivers the best outcome across every metric.

7. Experiment 4: Redis DECR vs Lua Script

7.1 Objective & Setup

Does a single atomic Lua script eliminate the DECR race window with acceptable overhead? The DECR approach has a small race window between going negative and the compensating INCR — another concurrent request can also go negative during this gap. The Lua script performs check-and-decrement atomically in one Redis call. Setup: 200 users, 100 stock, 60s, 2 instances, no resilience patterns.

7.2 Results

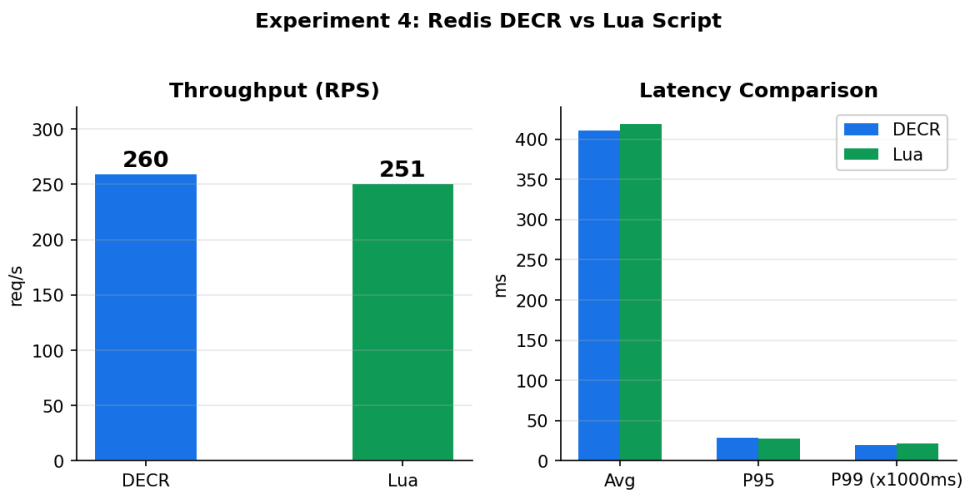

Figure 6: Experiment 4 — DECR vs Lua Script Comparison

Metric	DECR	Lua Script	Difference
Throughput (req/s)	260	251	-3.5%
Avg Latency (ms)	411	419	+1.9%
P95 (ms)	29	28	-3.4%
P99 (ms)	20,000	22,000	+10%
Correctness	PASS	PASS	Equal
Race Window	Yes (small)	No	Lua wins

Figure 7: Experiment 4 — Summary Table

7.3 Analysis

The Lua script adds ~8ms average latency and costs ~9 req/s throughput. However it eliminates the race window entirely — Redis guarantees Lua scripts execute atomically, so no other command can run between the check and the decrement. The DECR race window is small but real: under thousands of concurrent decrements, two requests can both observe a negative value before either INCR corrects it, creating a transient oversell condition.

Key takeaway: The Lua script provides a stronger correctness guarantee at only 3.5% throughput cost. For a production flash sale system where overselling has direct revenue impact, this trade-off is clearly worthwhile.

8. Conclusion

These four experiments validate core distributed systems principles with concrete, measurable evidence:

- **Horizontal scaling works, but needs shared state:** Adding instances behind an ALB maintains correctness, but in-memory counters cause 4x overselling. Redis DECR is a correctness requirement.
- **Correctness has a measurable but acceptable cost:** Redis introduces ~5% throughput reduction and ~40% P99 increase vs in-memory — negligible compared to the business impact of overselling.
- **Resilience patterns are worth the complexity:** Combining Fail Fast, Bulkhead, and Circuit Breaker delivers +73% throughput and -41% P99 under primary load. The patterns are complementary, each addressing a different failure mode.
- **Atomic operations matter at the boundary:** The Lua script eliminates the DECR race window at only 3.5% throughput cost — a clear win for production correctness.

9. Future Work

- **Experiment 5 — SQS Async Decoupling (implemented, not yet load-tested):** POST /purchase returns 202 Accepted immediately and enqueues to SQS. A background worker processes stock deductions asynchronously. Expected to dramatically improve throughput by decoupling intake from processing — similar to the async pattern validated in HW7.
- **Distributed Locust:** Run load generators across multiple machines to remove the ~1,500 RPS client CPU ceiling observed in Experiments 1 & 2 and observe true server-side scaling.
- **Auto-scaling:** Replace manual Terraform variable changes with CloudWatch-triggered ECS auto-scaling policies responding to real-time load.
- **Observability:** AWS X-Ray distributed tracing across ALB, ECS instances, and Redis; CloudWatch alarms on P99 latency and error rate thresholds.